ChatGPT

Pickaxe Capital Bookmark Intelligence Plan

Opportunity and core diagnosis

Executive summary

The opportunity is real, but it is not “31,533 bookmarks = instant edge.” The actual opportunity is that you already own a very large **lead archive** that can be turned into a **decision-support system**: a local, manually curated engine that helps you surface sources, capture lessons, route signals, and turn scattered links into reusable research assets. Your own brief frames the goal correctly: organized intelligence for better decisions, website building, knowledge organization, and CEO B review, not guaranteed wealth or automated trading. filecite turn0file1

The most important finding is structural: this archive is dominated by **social discovery**, not by stable knowledge objects. Your project brief and uploaded intelligence-map visuals summarize the archive at roughly **31,533 bookmarks across 914 folders**, with the top category labeled **Social / X / Sentiment**; a local parse of the uploaded Chrome bookmarks HTML confirms the same pattern, with **twitter.com, x.com, and instagram.com totaling 24,687 bookmarks**, or about **78%** of the archive. That means the bookmark system should be designed first as a **triage and routing engine**, not as a static library.
filecite turn0file1 filecite turn0file0

A second critical finding is that the archive is not mainly suffering from literal duplication. In the uploaded HTML export, exact duplicate URLs are low relative to total volume, while the bigger problem is **context quality**: many bookmarks are individual social posts, profile pages, search-result pages, and fast-decaying leads. In other words, the right question is not just “How do I dedupe this?” but “Which bookmarks deserve to become sources, lessons, watchlist items, or product tasks?” filecite turn0file0

A third finding is that your archive already contains the seeds of a differentiated founder OS. The uploaded HTML includes a small but strategically important cluster of **local prototypes and product pages** such as options dashboards, research desks, MaxClaw/Pickaxe pages, and local file URLs. Those are not “sources” in the same sense as public market research links; they are **product DNA** and should become a separate operating layer inside the system. filecite turn0file0

The build order that is actually worth doing first is straightforward:

Priority	Build first	Why it matters
Highest	Manual HTML import, parsing, folder tree, domain stats, page-type detection, trust labels, routing to Source Hub / Archive / Signals / CEO B Review	This solves the real bottleneck: triage and organization

Priority	Build first	Why it matters
High	Archive notes, decision notes, source cards, signal cards, mission queue	This converts raw bookmarks into reusable intelligence
High	Score model, duplicate checks, search-page detection, social-post handling	This prevents junk from polluting the system
Medium	Domain-level source monitoring, dossier views, founder/product idea routing	This increases repeat value
Later	Charts/workspaces, export snapshots, richer analytics	Useful only after categories and triage work
Future Adapter only	Live market feeds, SEC/FRED/API connectors, YouTube API, any social/API integrations	Adds complexity and compliance burden too early

What the archive actually contains

Your pasted brief says the archive is approximately **31,533 bookmarks**, **914 folders**, with major buckets around **24,740 Social / X / Sentiment**, **1,251 Markets / Trading**, **478 Video / Learning**, **63 AI / Coding**, **37 Local Codex Projects**, and **4,964 Other**. The uploaded map images visually reinforce the same summary and top-domain list. 【filecite】turn0file1【】

A local parse of the uploaded HTML export sharpens those numbers. The top domains are led by **twitter.com (22,354)**, **instagram.com (1,950)**, **google.com (935)**, **youtube.com (478)**, **x.com (383)**, then a much smaller long tail of market and research sites such as **investopedia.com**, **tradingview.com**, **barchart.com**, **unusualwhales.com**, **zerohedge.com**, **investing.com**, and **sec.gov**. That confirms that the archive is overwhelmingly a **lead-discovery corpus**, not a balanced source library. 【filecite】turn0file0【】

The page-type breakdown matters even more than the domain list. In the uploaded HTML, the Twitter/X collection is primarily composed of **individual post URLs** rather than evergreen pages, while Google bookmarks are mostly **search result pages** rather than final destination resources. That means a very large share of the archive should not be treated as durable knowledge by default. Social posts belong in a **lead queue** unless and until they are converted into structured notes, source cards, or research tasks. Search-result pages are mostly retrieval artifacts and should usually be discarded after extraction. 【filecite】turn0file0【】

The folder structure also tells an important story. A large portion of the archive lives under acquisition-style paths such as **Likes & Bookmarks**, **Likes: Monthly**, **Likes: MM/DD**, and similar ingest folders. That is useful provenance, but poor information architecture. You should preserve original path metadata for auditability, but you should **not** let the imported Chrome folder tree become the long-term information architecture of the product. 【filecite】turn0file0【】

The upshot is simple: the archive is valuable, but its value comes from **routing and enrichment**. Social discovery stays valuable only if it is turned into **source trust**, **archive notes**, **watchlist items**, **decision**

memos, or **product tasks**. Otherwise it remains a giant pile of recency-biased signals.
filecite turn0file1 filecite turn0file0

Taxonomy and scoring

Bookmark Intelligence taxonomy

The cleanest taxonomy is not identical to the original bookmark folders. It should reflect **how a bookmark will be used** inside Pickaxe Capital.

Category	What belongs here	Primary destination	Default action
Market Intelligence	Market dashboards, options/flow tools, macro calendars, market news readers	Source Hub	Keep as active source if credible
Company and Stock Research	SEC filings, exchange pages, IR pages, earnings resources, company dossiers	Source Hub + Archive Vault	Promote to dossier if repeatedly used
Macro and Policy	FRED, economic series, rates, policy, geopolitics, public data	Source Hub + Archive Vault	Archive with tags and thesis notes
Social Sentiment Leads	X/Twitter posts, Instagram posts, profile pages, follow graphs	Signals / Watchlist	Treat as leads, not facts
Trading Education	Investopedia, structured explainers, books, tutorials, YouTube learning	Archive Vault	Extract checklists and lessons
AI and Coding	OpenAI, Anthropic, GitHub, docs, tools, agent frameworks	Source Hub + Archive Vault	Build notes and tool cards
Website and Product Lab	file:// URLs, localhost pages, Pickaxe prototypes, dashboards	Product backlog / Mission Queue	Separate from public research
Founder and Business Building	founder ops, growth, distribution, design, workflow tools	Source Hub + Archive Vault	Turn into operator playbooks
Archive and Long-Term Knowledge	evergreen references, PDFs, books, theme studies	Archive Vault	Summarize and retain
Personal and Life	travel, logistics, personal learning, utilities	Personal Vault	Keep outside core research views
System and Retrieval Junk	google search pages, bookmarks pages, chrome pages, login pages	Ignore / Cold archive	Usually exclude from intelligence views

Category	What belongs here	Primary destination	Default action
Low Trust and Noise	rumor-heavy pages, low-context reposts, unverifiable hot takes	Ignore or lead-only	Never elevate without verification

This taxonomy is a better fit for your goals than the raw folder tree because it matches your requested operating layers: **Source Hub**, **Archive Intelligence Vault**, **Signals / Watchlist**, **CEO B Review**, and product-building inside the Pickaxe site. filecite turn0file1

Which categories should power which system

Your system should split by **operational role**, not only by topic.

System	Best input categories	What it should contain
Source Hub	Market Intelligence, Company and Stock Research, Macro and Policy, AI and Coding, Founder and Business Building	Domain cards, source notes, trust labels, tags, “why this source matters”
Archive Intelligence Vault	Trading Education, Archive and Long-Term Knowledge, curated company/macro notes, founder playbooks	Summaries, lessons, checklists, decision notes, versioned snapshots
Signals / Watchlist	Social Sentiment Leads, market dashboards, catalyst pages, selected macro/earnings links	Signal cards, watchlists, catalyst dates, confidence, review status
Product Lab	Website and Product Lab, AI and Coding, founder ops links	Prototype references, UI ideas, build tasks, design inspirations
Personal Vault	Personal and Life	Deliberately separated from core operating system
Ignore / Cold storage	System and Retrieval Junk, Low Trust and Noise	Retained only for provenance if needed

Source quality scoring model

Your scoring system should decide **what a bookmark becomes**, not just how “good” it looks. Use a 0–100 model with positive components and explicit penalties.

Component	Range	What it measures
Usefulness	0–15	Does this help you do real work repeatedly?
Credibility	0–15	Primary/official source vs hearsay/opinion/anonymous social
Repeat value	0–10	Worth revisiting more than once?
Signal potential	0–10	Useful for alerting, catalysts, narratives, sentiment

Component	Range	What it measures
Archive value	0–10	Worth preserving as a long-term knowledge object
Market relevance	0–10	Relevant to investing research workflow
Founder relevance	0–10	Relevant to building Pickaxe Capital / AI Habitat OS
Freshness	0–5	Timeliness matters for this source
Uniqueness	0–5	Adds something not already in the archive
Duplicate penalty	0 to -10	Exact/canonical duplicate or near-duplicate
Low-trust penalty	0 to -15	Rumor-heavy, unverifiable, manipulative, context-poor
Retrieval-junk penalty	0 to -10	Search pages, login pages, routing pages, dead context

A practical formula is:

```
score =
usefulness + credibility + repeat_value + signal_potential + archive_value +
market_relevance + founder_relevance + freshness + uniqueness
- duplicate_penalty - low_trust_penalty - retrieval_junk_penalty
```

Use this routing threshold:

Score band	Meaning	Default destination
85–100	Core source	Source Hub
70–84	Strong working asset	Source Hub or Archive Vault
55–69	Useful but incomplete	Archive with note or Research Task
35–54	Weak lead	Signals queue or Ignore
0–34	Noise / junk / low trust	Ignore or Cold archive

The scoring model should also assign a **trust label** independent of the numeric score:

Trust label	Typical examples	Default handling
Primary	SEC, exchange pages, company IR, official docs	High credibility boost
Analytical	strong research tools, reference sites, credible explainers	Medium credibility
Secondary media	commentary and news aggregators	Useful, but verify before acting

Trust label	Typical examples	Default handling
UGC / social	X posts, Instagram posts, Reddit threads	Lead-only unless verified
Retrieval artifact	search pages, login pages, folders, internal utility pages	Usually exclude

The reason to weight credibility so heavily is visible in your archive itself: the social layer is enormous, while primary sources are comparatively scarce. In the uploaded HTML, **SEC bookmarks are tiny compared with Twitter/X volume**, so the system must intentionally over-reward primary evidence or the archive will remain socially noisy. filecite turn0file0

Product architecture

Top Source Hub plan

Your Source Hub should be a **curated source universe**, not a mirror of the raw bookmark archive. Each source needs a source card with: domain, category, trust label, notes, why it matters, recurring use cases, and linked archive items.

Source Hub section	Contents	Keep active when
Trusted Market Sources	SEC, exchanges, company IR, earnings and filings sources	Source is primary or near-primary
Market Tools and Dashboards	TradingView, Barchart, options/flow tools, screeners	You use it weekly or it fills a real workflow gap
Macro and Data	FRED, economic series, calendars, public macro datasets	It supports repeat macro analysis
Social Signal Monitors	handpicked profiles, watch accounts, theme clusters	Only if they repeatedly generate useful leads
Trading Education	high-quality explainers, books, structured tutorials	It produces reusable lessons/ checklists
AI and Build Stack	GitHub repos, OpenAI/Anthropic/ OpenJarvis docs, coding tools	It helps build the product
Founder Operating System	growth, copy, design, workflow, product strategy	It improves execution

Do **not** make Source Hub a dump of every bookmarked domain. For example, Google search URLs should almost never become source cards, and most single social posts should not either. This matters because your uploaded HTML shows that many links are retrieval scaffolding rather than real sources. filecite turn0file0

Archive Intelligence Vault plan

The Archive Vault should turn bookmarks into **knowledge objects**. Every archived item should answer: *what this is, why it matters, what I learned, what I should do next, and what decision it influenced.*

A strong Archive Vault record should include:

Field	Purpose
Title	Clean human-readable title
Source URL	Original bookmark
Original folder path	Provenance only
Domain and category	Retrieval and filtering
Summary	What the source says
Why saved	Why it matters to Pickaxe / CEO B
Trust label	Primary / analytical / secondary / UGC
Key takeaways	Reusable lessons
Checklist / playbook	Actionable operational notes
Linked tickers / themes / projects	Cross-linking
Decision impact	What decisions it informed
Future tasks	Follow-up queue
Snapshot date	When this was captured
CEO B note	Final human judgment

The rule should be: **nothing enters the Vault raw**. If a bookmark is not worth at least a brief summary or note, it probably belongs in Source Hub, Signals, or Ignore instead.

Signals and watchlist plan

Your market-related bookmarks should flow through a decision-support path like this:

```
Bookmark import
→ page-type detection
→ scoring + trust label
→ if social/lead-like, create Signal Candidate
→ manual validation
→ chart / catalyst / source check
```

```
→ CEO B Review
→ Archive / Watchlist / Ignore
```

Operationally, use these roles:

Layer	Role
Signal Scout	collects candidate ideas, narratives, anomalies, unusual setups
RK Tracker	tracks repeat keywords, names, tickers, and catalyst patterns
Market Chart Workspace	manual visual confirmation, context checks, levels, timing notes
CEO B Review	decision gate: ignore, archive, research deeper, productize, or monitor

Each signal card should contain: ticker/theme, source URL, source trust, catalyst type, sentence summary, why it may matter, counterpoint, expiry date, and next review date. This aligns with your requirement to avoid auto-trading and keep every action under **CEO B Review**. filecite turn0file1

CEO B Review system

This should be the human control layer for everything important. Every item should end in exactly one of these states:

CEO B outcome	Meaning
Ignore	no value or too low trust
Archive	worth preserving as knowledge
Source Hub	worth monitoring as a recurring source
Signal Task	possible market lead to monitor
Research Task	requires validation or deeper work
Product Idea	could become a website/tool feature
Content Idea	could become a post, memo, lesson, or publishable asset
CEO B Decision	decision recorded with rationale

The review screen should force these fields before finalization: **confidence, trust label, why it matters, next step, and owner**. That reduces impulsive escalation from social content.

Bookmark Miner specification

What the `/bookmarks` page should do

The safest and highest-leverage MVP is a **manual, local-first bookmark miner** that ingests the Chrome HTML export and never scrapes the web. A browser can do this safely because `FileReader` can asynchronously read files the user explicitly selects, and it cannot read arbitrary local files by pathname without user action. [1]

Use this feature plan:

Feature	Priority	Notes
HTML import	Must have	Manual local import only
Parser and folder tree	Must have	Preserve original hierarchy as provenance
Top domains and category charts	Must have	Quick diagnosis of archive shape
Page-type detection	Must have	detect social posts, profiles, searches, local files, docs
Duplicate detection	Must have	exact + canonical URL
Domain trust labels	Must have	primary / analytical / secondary / UGC / junk
Category filters	Must have	taxonomy-based, not raw folders only
Bookmark score	Must have	0–100 routing score
Send to Archive	Must have	creates archive item draft
Send to Source Hub	Must have	creates or links to source card
Send to Signals	Must have	creates signal candidate
Send to RK Tracker	Must have	keyword/theme/ticker tracking
Send to CEO B Review	Must have	places item in mission queue
Create task	Must have	research or product follow-up
Search and saved filters	Must have	daily usability
localStorage support	Must have	filters, view preferences, recent actions
Export JSON snapshot	Should have	backup and portability
Screenshot/notes fields	Should have	useful for archive quality
Live sync	Do not build now	keep disabled

Feature	Priority	Notes
Scraping	Do not build	explicitly out of scope

What should be built locally before any integrations

Before you add any live adapters, build these manual workflows:

Workflow	Why it comes first
Import and triage	proves the taxonomy and scoring model
Source-card creation	builds the recurring source layer
Archive-note creation	converts noise into knowledge
Signal validation queue	prevents social leads from bypassing review
Weekly review digest	creates operating rhythm
Product/backlog routing	turns local prototypes and ideas into execution
Snapshot export	preserves state without cloud dependence

This order is critical because your maps and brief already emphasize **manual import**, **local file only**, **no live sync**, **no scraping**, and **CEO B Review**. That is a good MVP boundary. filecite turn0file1

Data model

Use a clean local model like this:

Entity	Core fields
`bookmarkItem`	`id`, `title`, `url`, `canonicalUrl`, `domain`, `path`, `dateAdded`, `pageType`, `category`, `tags`, `trustLabel`, `score`, `status`, `notes`
`folder`	`id`, `name`, `parentId`, `fullPath`, `bookmarkCount`
`domain`	`domain`, `category`, `trustLabel`, `bookmarkCount`, `notes`, `sourceHubStatus`
`category`	`id`, `name`, `description`, `defaultDestination`
`sourceScore`	`bookmarkId`, `usefulness`, `credibility`, `repeatValue`, `signalPotential`, `archiveValue`, `marketRelevance`, `founderRelevance`, `freshness`, `uniqueness`, `penalties`, `finalScore`
`archiveItem`	`id`, `bookmarkId`, `summary`, `takeaways`, `checklist`, `linkedEntities`, `decisionImpact`, `version`, `createdAt`

Entity	Core fields
`signalTask`	`id`, `bookmarkId`, `ticker`, `theme`, `catalyst`, `confidence`, `expiryDate`, `reviewStatus`, `ceoDecision`
`ceoReviewItem`	`id`, `bookmarkId`, `recommendedAction`, `confidence`, `whyItMatters`, `nextStep`, `finalDecision`, `reviewedAt`
`missionQueueItem`	`id`, `type`, `linkedId`, `priority`, `owner`, `status`, `dueDate`

Codex implementation plan for a static Node app

A browser-based static site is capable of the core MVP. MDN documents that Web Storage persists by origin and that `localStorage` is synchronous and string-only, while IndexedDB is meant for larger structured datasets. MDN also documents a maximum Web Storage budget of about **10 MiB total per origin**—roughly **5 MiB localStorage** and **5 MiB sessionStorage**—which is too small and too blocking for a richly enriched 31k-bookmark corpus. So the right design is: **localStorage for UI state and small queues; IndexedDB for corpus persistence if you persist the full enriched archive; JSON export for backup.** [2]

Use your existing file layout like this:

File	Responsibility
`public/index.html`	dashboard shell, import area, filters, tables, detail drawer, review modal
`public/app.js`	parser, classifiers, scorers, routing engine, render functions, imports/exports
`public/styles.css`	layout, tables, queue colors, trust labels, score bands
`public/habitat-data.js`	taxonomy constants, domain rules, trust maps, score weights, starter tags
`server.mjs`	static file server only; no live scraping, no cloud sync, no secret API keys

Recommended build phases:

Phase	Outcome
Phase one	import HTML, parse bookmarks, show stats, tree, domains, top categories
Phase two	page-type detection, score engine, trust labels, basic dedupe
Phase three	Archive / Source Hub / Signals / CEO B Review routing
Phase four	task queue, JSON export, local persistence, saved filters
Phase five	dossier views, domain cards, archive notes, review workflows

If you want optional local encryption later, the browser does provide `SubtleCrypto`, but MDN explicitly warns that it is low-level and easy to misuse. So if you add "encrypted vault," label it **experimental** until the design is reviewed carefully. [3]

What should stay future-adapter only

Keep these out of the MVP:

Future adapter	Why not now
SEC EDGAR live ingestion	Good future source, but SEC notes its data APIs do not support CORS, which makes direct browser-only integration awkward for a static app; better as a later adapter or mediated service. [4]
FRED macro ingestion	Valuable, but still an integration layer rather than a triage-layer priority. The FRED API is well suited to future programmatic macro retrieval. [5]
YouTube live enrichment	Officially possible through the YouTube Data API, but it requires Google project setup, API enablement, auth patterns, and quota management. [6]
Social platform integrations	Keep manual and review-led unless you later design explicit, compliant API-based flows
Live broker/data vendor connections	Too much compliance and scope risk for this stage
Auto-execution or trading automation	Explicitly outside your safety boundary

Value, risk, and roadmap

Money and opportunity map

This system can create value, but mostly by improving **process quality**, not by conjuring guaranteed returns. The realistic opportunity map looks like this:

Value path	Realistic benefit
Better investment research process	less scattered discovery, better provenance, stronger source discipline
Source monitoring discipline	know which domains/accounts actually earn attention
Content engine	convert archive notes into posts, memos, lessons, dashboards, explainers
Paid research products	only after you have repeatable internal dossiers and reviews
Education system	reuse saved lessons, checklists, and annotated reading

Value path	Realistic benefit
AI tool directory	organize build tools and docs into a founder/coding stack
Market playbook archive	build reusable setup notes, catalyst checklists, and review templates
Personal decision OS	separate personal/logistics bookmarks from capital-intelligence workflows
Founder operating system	use the same routing engine for tools, ideas, prototypes, and decisions

The value thesis is strongest in areas where the archive becomes **repeatable infrastructure**: source cards, decision notes, playbooks, and backlog routing. It is weakest where the archive remains a passive pile of links.

Risks, compliance issues, and bad ideas to avoid

Your own safety rules are well chosen: no guaranteed money claims, no direct trading advice, no auto-trading, no scraping protected/login-gated sites, no assumption that social posts are true, and CEO B review before execution. Those should remain hard constraints. filecite turn0file1

The bad ideas to avoid are these:

Bad idea	Why it is bad
Treating bookmarked posts as facts	social links are leads, not verified evidence
Promoting “alpha” or guaranteed outcomes	weakens credibility and adds regulatory/commercial risk
Auto-routing social content directly into action	bypasses your stated human review control
Building around Google searches as sources	search-result pages are retrieval artifacts, not knowledge assets
Letting Chrome folders define product IA	imported folder trees reflect acquisition habits, not durable knowledge design
Storing the whole enriched corpus in localStorage	MDN documents localStorage as synchronous, string-only, and quota-limited; it is not a good full-database layer. [7]
Claiming strong encryption casually	MDN warns SubtleCrypto is easy to misuse. [3]
Going live with APIs too early	adds auth, quota, terms, and maintenance before the taxonomy is proven

Bad idea	Why it is bad
Mixing product prototypes with public research	different assets, different workflows

The 30/60/90 day plan

Timeframe	What to do	Concrete outcome
Week one	Build HTML importer, parser, folder tree, domain counts, page-type detection, trust labels, score skeleton	You can see the archive clearly
Week two	Add category filters, Archive/Source/Signals/CEO B actions, review modal, JSON export	You can triage bookmarks instead of just viewing them
Month one	Promote top 50 domains into source cards; archive 100 high-value items with notes; create 25 signal cards; clean junk filters	First working intelligence layer exists
Month two	Build company/theme dossiers, founder/product backlog routing, weekly review workflow, saved views, score tuning	Archive becomes operationally useful
Month three	Add mission queue, dashboard summaries, archive search, review analytics, optional IndexedDB persistence	System becomes a real local command center

A more specific milestone plan:

Period	Success condition
30 days	Bookmark Miner works locally and routes items correctly
60 days	Source Hub and Archive Vault contain enough curated material to replace ad hoc browsing
90 days	CEO B Review and Signals workflow create a repeatable weekly operating rhythm

Open questions and limitations

This analysis is high-confidence on the core structure, but it has boundaries. I had the uploaded Chrome bookmarks HTML, the pasted project brief, and the two intelligence-map images. I did **not** have retrievable access to any additional CSV/JSON summaries, AGENTS.md, PROJECT_STATUS.md, or other project files referenced in the brief, so those were not incorporated. Where I refer to exact page-type counts or duplicate counts, that is based on a local parse of the uploaded bookmark HTML. filecite turn0file0 filecite turn0file1

Final Codex prompt

Use this as the build prompt for Codex:

```
You are adding a local-first Bookmark Miner / Bookmark Intelligence system to my
existing Pickaxe Capital static Node website.

Project constraints:
- Do not migrate frameworks.
- Do not add a backend database.
- Do not add cloud sync.
- Do not scrape X, Twitter, Instagram, YouTube, or any protected/login-gated
site.
- Do not add any live market or broker integrations.
- Do not add auto-trading or trading execution.
- Every important action must route through CEO B Review.
- This is a decision-support and knowledge-organization tool, not a guaranteed-
money tool.

Tech/files to use:
- public/index.html
- public/app.js
- public/styles.css
- public/habitat-data.js
- server.mjs

Build a /bookmarks experience with these capabilities:
1. Manual import of a Chrome bookmarks HTML export using browser FileReader
only.
2. Parse folders and bookmarks into a local in-browser data model.
3. Preserve original folder path as provenance metadata.
4. Compute:
   - total bookmarks
   - total folders
   - top domains
   - page types (social post, profile, search, local file, doc, video, general
page)
   - duplicate URLs (exact and canonical)
5. Add category taxonomy:
   - Market Intelligence
   - Company / Stock Research
   - Macro / Policy
   - Social Sentiment Leads
   - Trading Education
   - AI / Coding
   - Website / Product Lab
   - Founder / Business
   - Archive / Long-Term Knowledge
   - Personal / Life
   - System / Retrieval Junk
   - Low Trust / Noise
```

```
6. Add trust labels:
   - Primary
   - Analytical
   - Secondary
   - UGC / Social
   - Retrieval Artifact
7. Implement a 0–100 score model using:
   usefulness
   credibility
   repeatValue
   signalPotential
   archiveValue
   marketRelevance
   founderRelevance
   freshness
   uniqueness
   duplicatePenalty
   lowTrustPenalty
   retrievalJunkPenalty
8. Add actions on each bookmark:
   - Send to Archive
   - Send to Source Hub
   - Send to Signals
   - Send to RK Tracker
   - Send to CEO B Review
   - Create Research Task
   - Create Product Idea
   - Create Content Idea
   - Ignore
9. Build these views:
   - Overview dashboard
   - Folder tree
   - Domain explorer
   - Bookmark table
   - Source Hub
   - Archive Vault
   - Signals / Watchlist
   - CEO B Review queue
   - Mission queue
10. Add filters for:
   domain, category, trust label, score range, page type, queue status
11. Add search across title, url, domain, tags, notes.
12. Add notes fields for archive and review:
   summary, whySaved, takeaways, checklist, linkedTickers, linkedThemes,
decisionImpact, nextStep
13. Use localStorage for:
   - UI preferences
   - saved filters
```

```
   - selected tabs
   - recent actions
   - small queue drafts
14. If persistent storage is needed for the parsed corpus, use browser IndexedDB
only, not a backend.
15. Add JSON export/import for local backup.
16. Keep styling consistent with Pickaxe Capital / AI Habitat OS.
17. Add clear labels:
   - Local file only
   - Manual import
   - No live sync
   - No scraping
   - CEO B Review required

Implementation guidance:
- Put schema constants, taxonomy rules, score weights, and trust maps in public/
habitat-data.js.
- Put parsing, classification, dedupe, scoring, rendering, and persistence logic
in public/app.js.
- Keep server.mjs as a simple static file server only.
- Use clean vanilla JS and modular functions.
- Make the UI usable with large bookmark sets.
- Preserve speed and readability over visual gimmicks.

Routing rules:
- Social posts default to Signals or Research Task, not Archive.
- Search result pages default to Ignore unless manually saved with notes.
- Local prototype URLs default to Product Lab / Mission Queue.
- Primary sources with high credibility can go to Source Hub or Archive.
- Nothing important should bypass CEO B Review.

Deliver a working static implementation with comments, clear function names, and
no framework migration.
```

---

1 https://developer.mozilla.org/en-US/docs/Web/API/FileReader
https://developer.mozilla.org/en-US/docs/Web/API/FileReader

2 7 https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

3 https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto
https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto

4 https://www.sec.gov/search-filings/edgar-application-programming-interfaces
https://www.sec.gov/search-filings/edgar-application-programming-interfaces

5 https://fred.stlouisfed.org/docs/api/fred/
https://fred.stlouisfed.org/docs/api/fred/

6 https://developers.google.com/youtube/v3/getting-started
https://developers.google.com/youtube/v3/getting-started